

CUAD Legal Clause Extraction

Fine-Tuning a Small Open Instruction Model for Structured Contract Review

Nolan Rink

Fine-Tuning a Small Open LLM for Structured CUAD Clause Extraction

Can LoRA fine-tuning improve structured CUAD extraction over zero-shot prompting and extractive QA?

0.9122

Fine-tuned found accuracy

whole-set on 8,493 test records

0.6798

Fine-tuned normalized F1

answer text overlap, whole-set

0.9408

Fine-tuned JSON parse rate

503 invalid outputs remain

Defense thesis

Fine-tuning helps schema reliability and detection, but positive-answer extraction and long-contract context remain the hard parts.

Structured output: category + found + normalized_answer + evidence_text

Contract review needs evidence-backed extraction

Legal review question

For a specific clause type, is the clause present in this contract context?

Not enough

A fluent model answer is weak if it cannot be parsed, checked, or tied to contract text.

Target behavior

Return a machine-readable decision plus the answer text and supporting evidence.

The project therefore treats CUAD as structured clause extraction, not broad legal advice or contract summarization.

Prior CUAD results are context, not a direct numerical baseline

Original CUAD benchmark

Span-extraction framing with AUPR and precision at fixed recall. Matching relies on confidence-ranked spans and a Jaccard-overlap rule.

No benchmark superiority claim

This project

Chunk/category input with JSON output. Invalid JSON counts as task failure in whole-set metrics.

No direct AUPR comparison

Fair conclusion

Use CUAD legal NLP work as motivation, but compare results only among the three methods evaluated here.

Controlled internal comparison

Scope: a reproducible single-GPU structured extraction pipeline

In scope

CUAD chunk/category extraction; TinyLlama LoRA fine-tuning; extractive QA and zero-shot baselines; whole-set metrics.

Out of scope

Full-contract retrieval, production legal workflows, model sweeps, Qwen runs, long-context modeling.

Why narrow scope helps

The project can make one clean experimental claim with reproducible artifacts instead of mixing incomplete infrastructure with model results.

Version-1 contribution

A complete experimental story: preprocessing, training, evaluation, figures, qualitative examples, and a final report.

CUAD supplies expert contract-review labels for clause extraction

41**CUAD clause categories**

present in each split

510**raw contracts**

408 train-side + 102 test

8,493**held-out test records**

chunk/category examples

Raw files used

train_separate_questions.json feeds

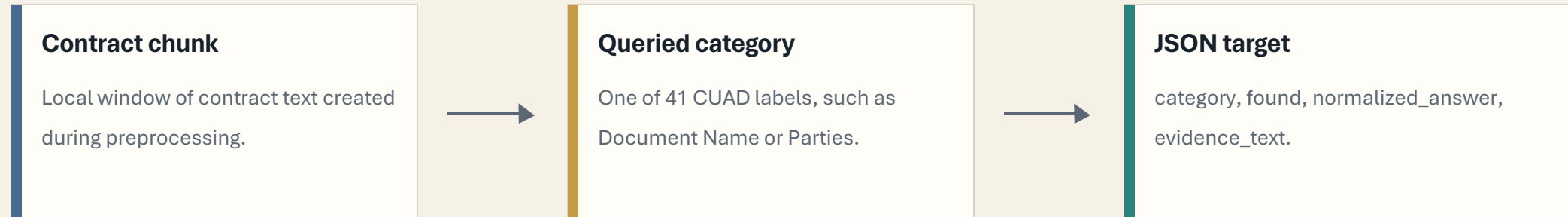
train/validation; test.json remains the held-out test source.

Purpose in this project

Working view

CUAD is converted into a supervised extraction task: each record asks whether one clause category appears in one contract chunk and, if so, what answer and evidence text should be returned.

Each example is one contract chunk paired with one CUAD category



Why no-answer records matter

A model that always extracts text would produce many false positives. A useful extractor must also learn when a category is absent from the chunk.

A single record asks for one structured answer

Contract chunk snippet

Exhibit 10.17 SUPPLY CONTRACT ... Contract No: ... The buyer/End-User:
Shenzhen LOHAS Supply Chain Management Co., Ltd.

Queried category

Document Name

Expected JSON

```
{
  "category": "Document Name",
  "found": true,
  "normalized_answer": "SUPPLY CONTRACT",
  "evidence_text": "SUPPLY CONTRACT"
}
```

found

present/absent decision

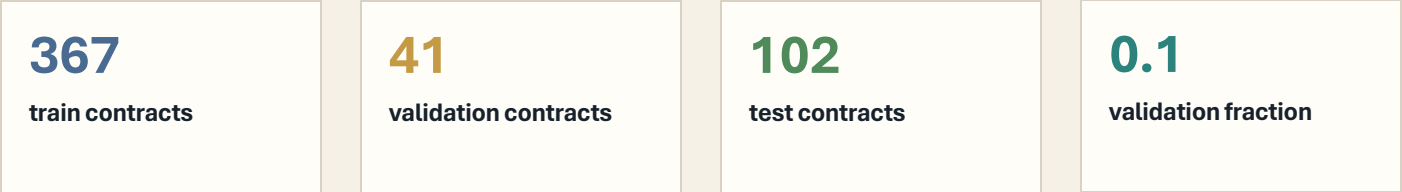
normalized_answer

short answer to compare

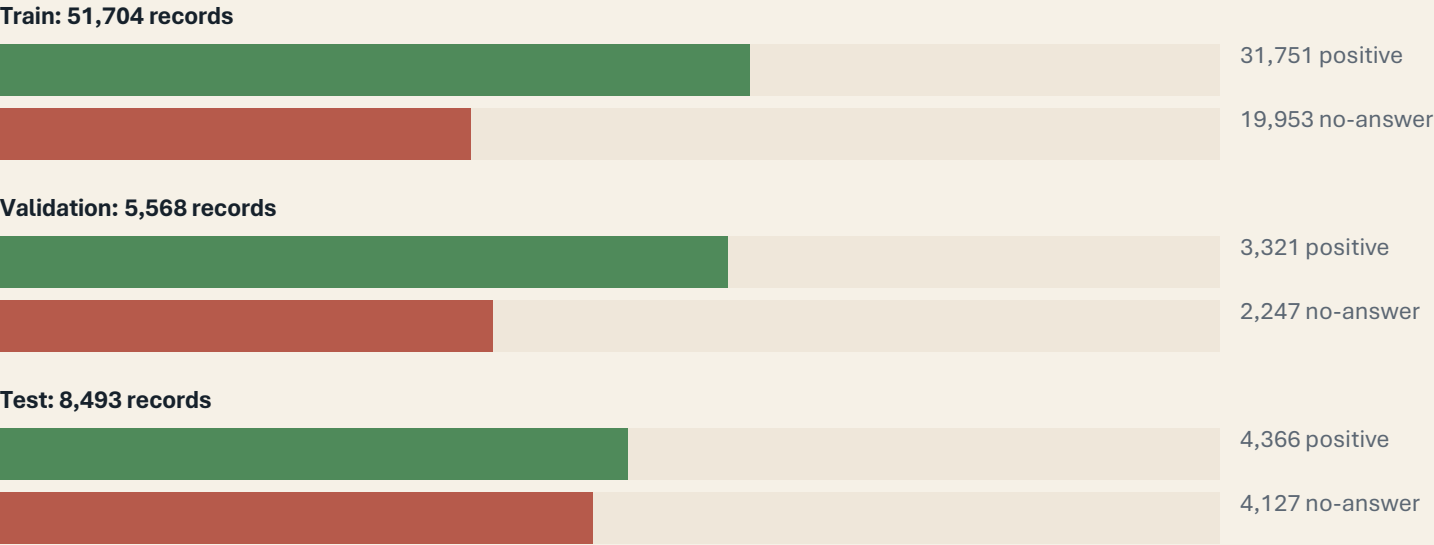
evidence_text

supporting contract text

The split is contract-level, with positives and no-answer cases in every split



Positive vs no-answer records



Test is nearly balanced: 4,366 answer-present records and 4,127 no-answer records.

Contract-level splitting keeps chunks from the same agreement in one split.

The task mixes detection, extraction, normalization, and formatting

Context is local

2,200-character chunks make training feasible, but they are not full-contract context.

No-answer is common

4,127 of 8,493 test records have no answer in the queried chunk.

Boilerplate is tempting

Party labels, dates, and headings can look like evidence for the wrong category.

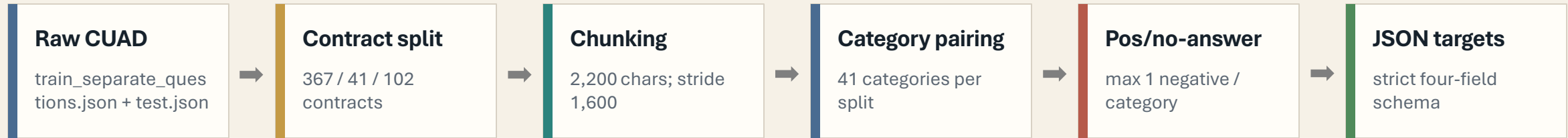
Normalization is hard

A multi-entity answer can be partly right but incomplete, especially for categories such as Parties.

JSON is a real constraint

For generative methods, an answer is unusable if it cannot be parsed into the expected schema.

Preprocessing turns raw CUAD files into model-ready JSON targets



Chunking keeps the run feasible on one GPU, but it also defines the evaluation boundary: this is not full-contract retrieval.

The supervised target depends on whether the answer is visible in the chunk

Positive record

Created when the category answer text or span is visible in the chunk. Target has found=true plus normalized answer and evidence text.

No-answer record

Created from chunks where the category answer is not visible. Target has found=false and null answer/evidence fields.

Sampling control

max_negative_chunks_per_category = 1 keeps negative records from dominating the dataset.

2,200

chunk size, chars

local contract window

1,600

stride, chars

overlap around boundaries

1

max negatives

per annotation/category

TinyLlama + LoRA made the experiment practical on one GPU

Base model

TinyLlama/TinyLlama-1.1B-Chat-v1.0

Fine-tuning method

LoRA adapters on attention and MLP projection modules.

Practical goal

Test structured extraction, not infrastructure scale.

q_proj

k_proj

v_proj

o_proj

gate_proj

up_proj

down_proj

2048

context window

1024

max source length

192

max target length

3,000

completed steps

0.0293

best eval loss

at step 2,400

Training maps a CUAD prompt into a strict JSON object

Prompt input

Instruction: return JSON only
Category: requested CUAD clause
Contract chunk: local text window
Constraint: exact schema keys



TinyLlama + LoRA

Small instruction model adapted
with parameter-efficient LoRA
adapters.



JSON target

```
{  
  category,  
  found,  
  normalized_answer,  
  evidence_text  
}
```

The schema is the learning target: invalid JSON, wrong found decisions, weak answer text, and weak evidence can be penalized separately.

Baselines test span extraction and prompting separately

Extractive QA baseline

deepset/bert-base-cased-squad2 predicts a span or no-answer. It is meaningful because CUAD resembles span extraction, but it does not naturally output the JSON schema.

Zero-shot generation

Same TinyLlama base model, no task-specific fine-tuning. Tests whether prompting alone can follow the schema.

384

max seq length

6,800

completed steps

1.1411

best eval loss

0.1644

parse rate

7,097

invalid JSON

out of 8,493

Success requires more than a plausible answer string

1. Parseable JSON

Output can be read as the expected object.

2. Correct found decision

Present versus absent is right.

3. Answer-present recall

Real answer-bearing records are recovered.

4. No-answer accuracy

Absent categories are correctly rejected.

5. Normalized answer F1

Returned answer overlaps the reference.

6. Evidence alignment

Support text points to the right contract language.

Whole-set metrics are primary because invalid JSON is a real task failure

Whole-set

All 8,493 test records. Invalid JSON counts as incorrect for generative methods.

Parsed-only

Diagnostic view restricted to outputs that parse.
Useful, but not the headline comparison.

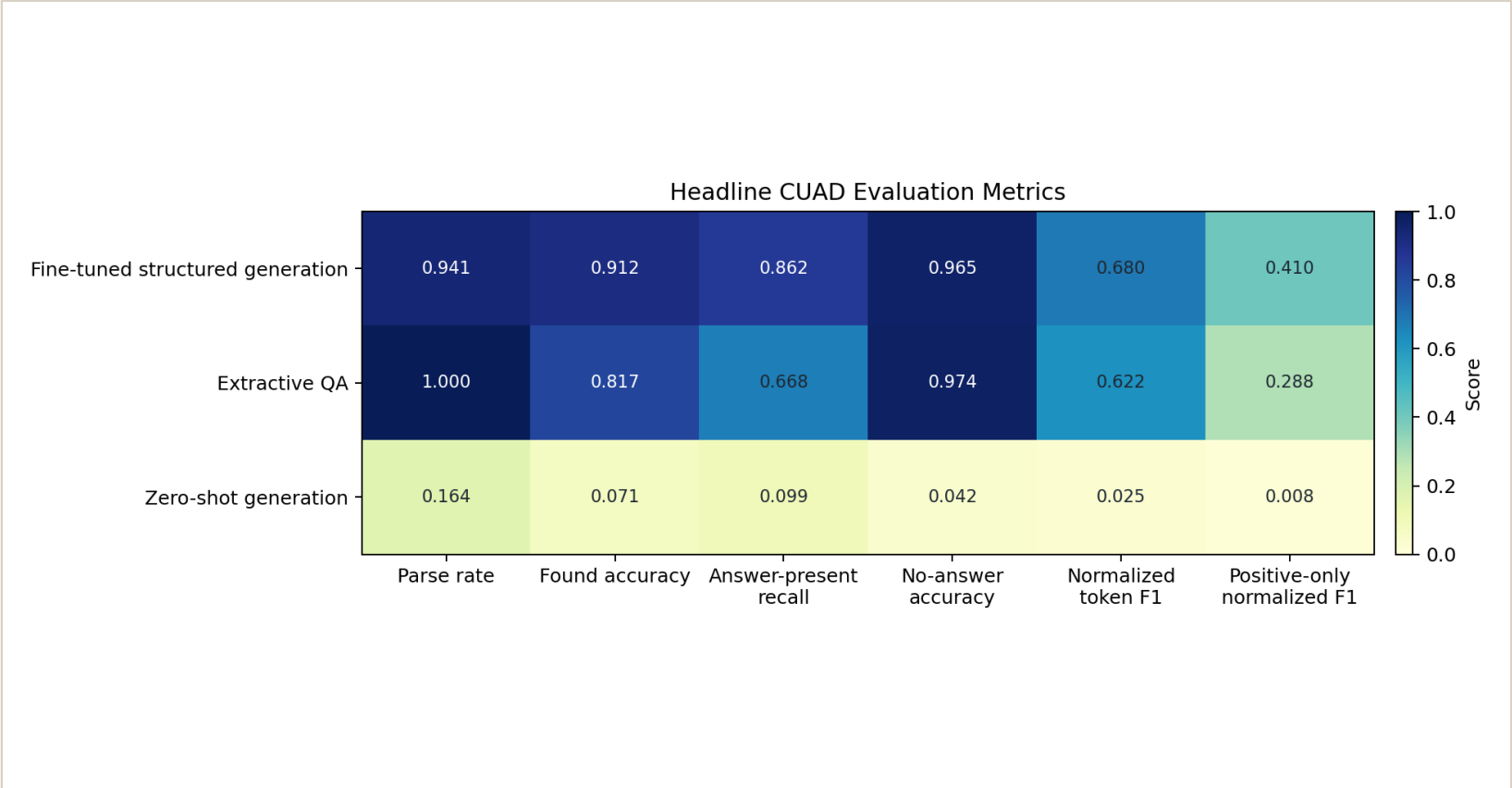
Positive-only F1

Restricted to answer-present references, so it isolates extraction quality from no-answer behavior.

Key metrics: parse rate, invalid count, found accuracy, answer-present recall, no-answer accuracy, normalized-answer F1, evidence token F1, per-category found accuracy.

This framing is also why original CUAD AUPR / Precision@Recall numbers are not direct baselines for this report.

Fine-tuned structured generation is strongest in the controlled comparison



Method metric scorecard from approved final artifacts

Main reading: fine-tuning improves schema reliability, found/not-found behavior, and answer overlap relative to the evaluated baselines.

Scope: these are whole-set chunk/category metrics, not full-contract retrieval or a direct CUAD-paper benchmark comparison.

The numbers tell three different method stories

Fine-tuned structured generation

Best overall. High JSON reliability and the strongest answer-present recall among the evaluated methods.

0.9122

found acc.

0.4098

pos-only F1

Extractive QA baseline

Strong no-answer behavior and competitive normalized F1 of 0.6216, but weaker recovery of answer-present cases.

0.8170

found acc.

0.2883

pos-only F1

Zero-shot generation

Mostly fails the structured output contract; invalid JSON dominates whole-set scoring.

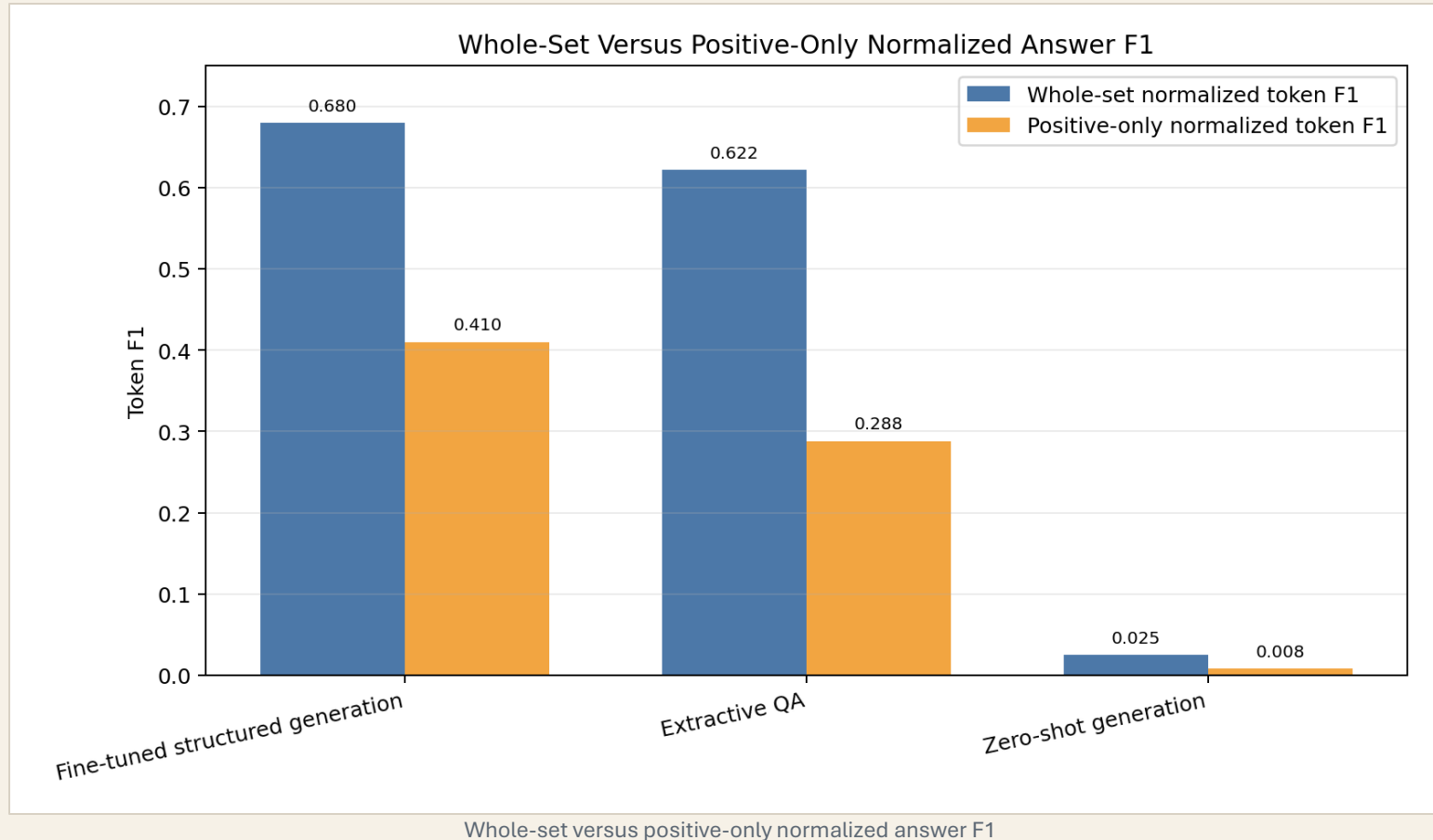
0.1644

parse rate

0.0082

pos-only F1

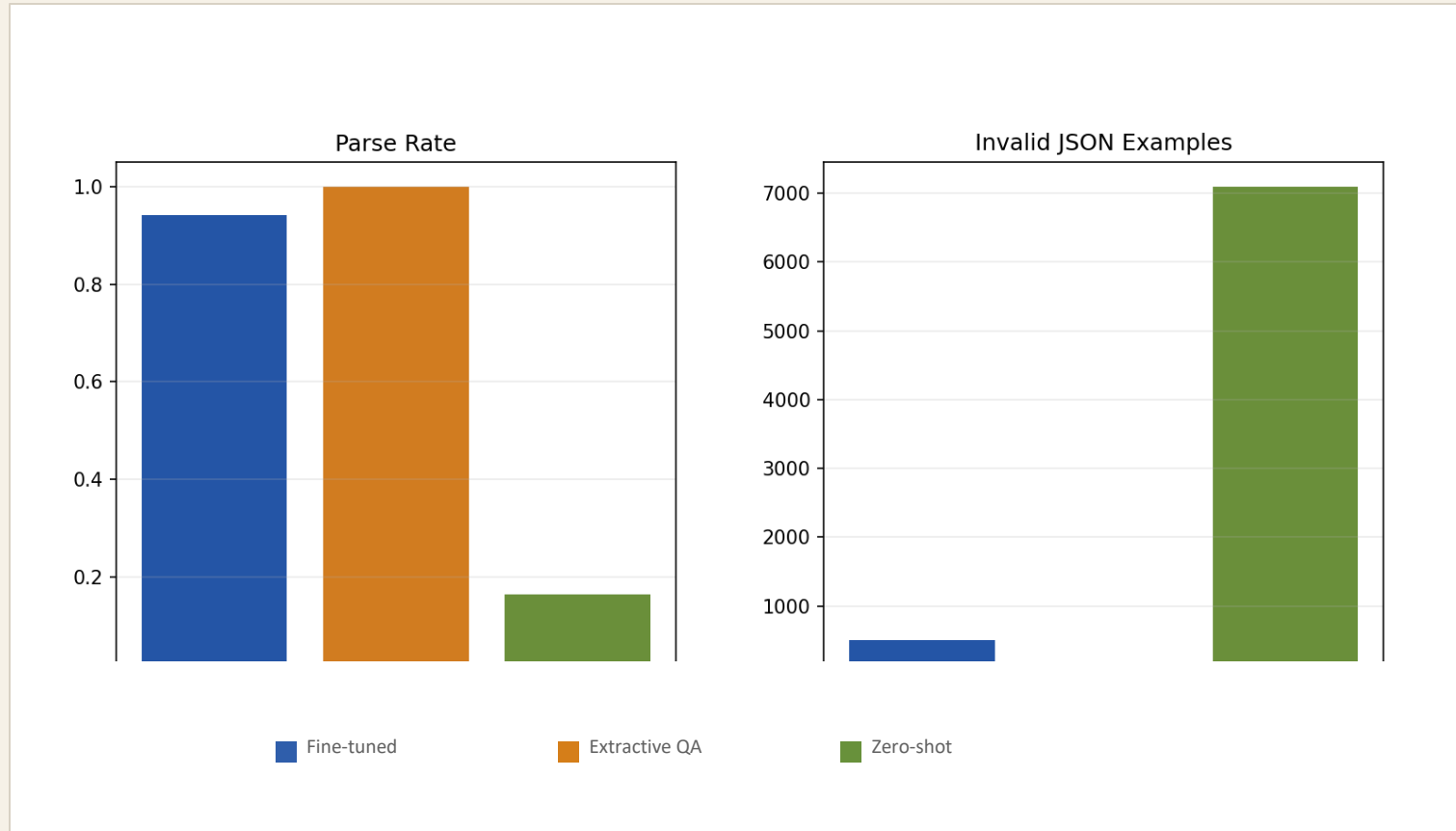
Whole-set F1 and positive-only F1 answer different questions



Whole-set F1 includes no-answer behavior and parse failures. Positive-only F1 isolates answer-present extraction quality.

The gap is the caveat: fine-tuning improves the structured task, but exact answer recovery remains hard.

Fine-tuning dramatically improves JSON reliability, but does not make it perfect



Parse rate and invalid JSON counts

503

fine-tuned invalid outputs

out of 8,493

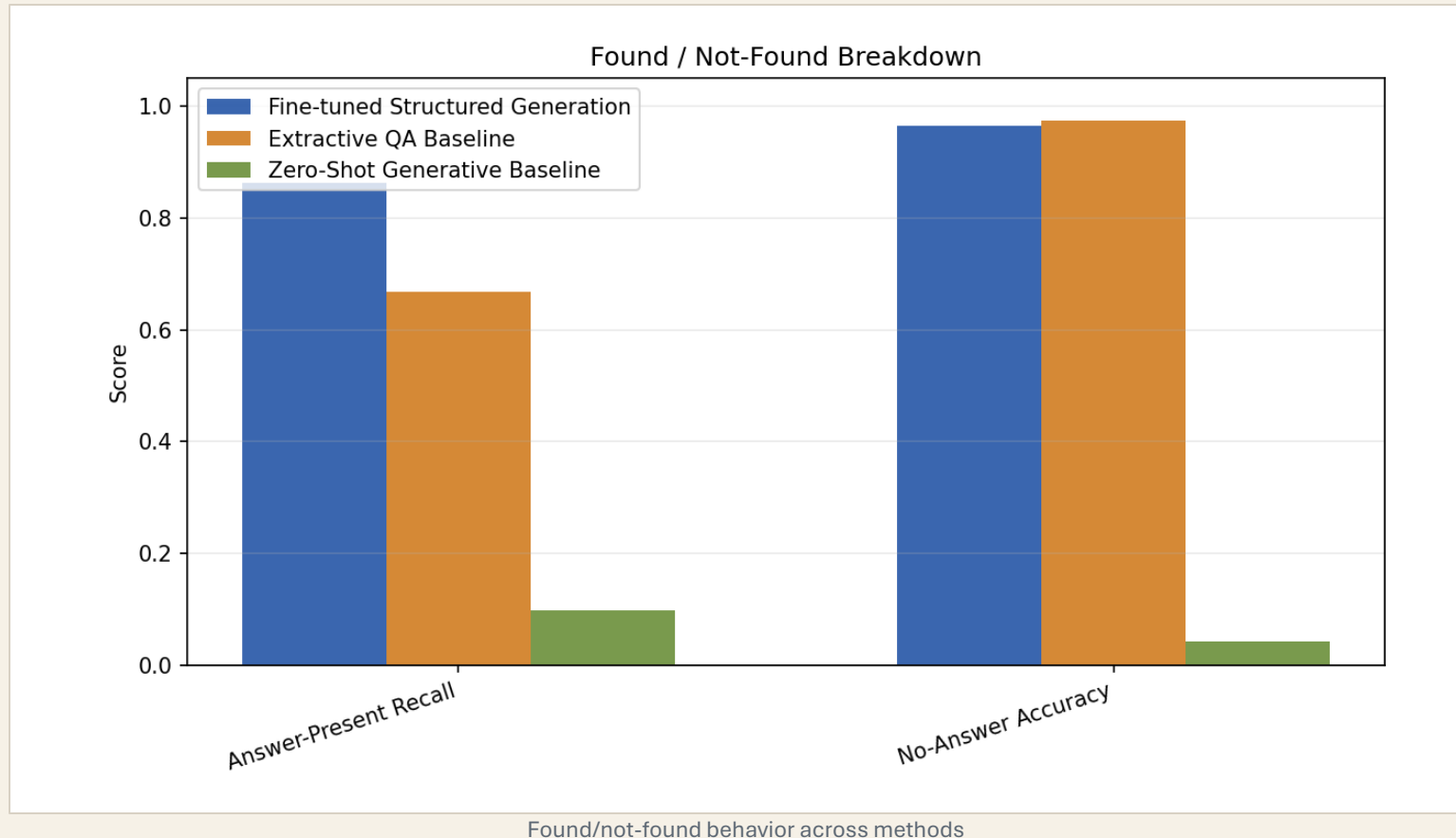
7,097

zero-shot invalid outputs

out of 8,493

For structured generation, format reliability is not cosmetic; it determines whether the output can be evaluated or used downstream.

Strong abstention and strong positive-case recovery are different behaviors



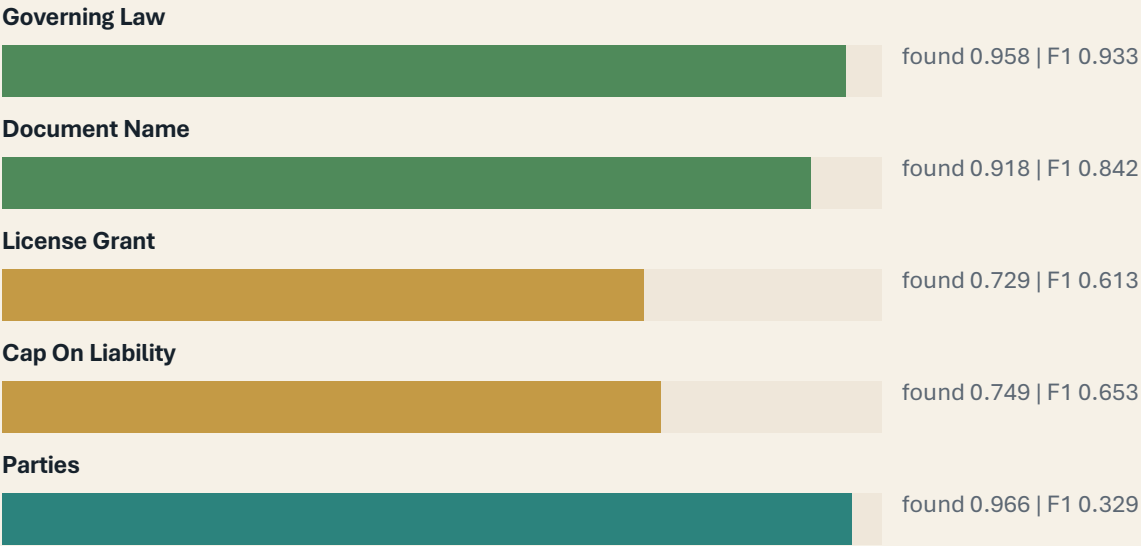
Fine-tuned model: answer-present recall 0.8619, no-answer accuracy 0.9654.

Extractive QA: no-answer accuracy is higher at 0.9743, but answer-present recall is lower at 0.6683.

Zero-shot fails both sides under whole-set scoring because invalid JSON is counted as failure.

Category-level variation is real, but the takeaway is simple

Fine-tuned structured generation, selected CUAD categories



How to read this

High found accuracy can still mask weaker answer text. Parties is the clearest case: detection is high, but normalized-answer F1 is much lower.

This is a diagnostic selection, not a statistical significance test.

Prediction-vs-reference snippets explain the remaining errors

Success: Document Name

Prediction: SUPPLY CONTRACT

Reference: SUPPLY CONTRACT

Behavior: parseable JSON and aligned evidence.

Partial: Parties

Prediction: Shenzhen LOHAS Supply

Chain Management Co., Ltd.

Reference: seller + buyer/end-user.

Behavior: real entity, incomplete normalized answer.

False positive: Parties

Prediction: Seller

Reference: no-answer

Behavior: boilerplate role label treated as evidence.

Zero-shot invalid

Output begins: Contract chunk:

Reference expects JSON.

Behavior: copied prompt text, parse failure.

Interpretation

The errors are not random: they include incomplete multi-entity normalization, category confusion from contract boilerplate, failure to abstain, and schema-following failure.

The result is useful, but bounded

Model capacity

TinyLlama is practical for one GPU, not the strongest possible legal extraction model.

Context boundary

Chunk/category evaluation is not full-contract retrieval or long-context reasoning.

JSON reliability

Fine-tuning reduced invalid outputs, but 503 fine-tuned outputs were still invalid.

Positive answers

Positive-only normalized F1 remains much lower than whole-set F1.

Baseline asymmetry

Extractive QA is meaningful, but it is not the same output format as JSON generation.

Legal use

This is an experimental graduate pipeline, not a production legal system.

The project clarified what to improve next

Lessons learned

1. Fine-tuning helps schema reliability.
2. Whole-set metrics can hide positive-answer difficulty.
3. Baselines need careful framing.
4. Legal extraction remains limited by context and normalization.

Future work

Qwen2.5-7B-Instruct LoRA/QLoRA as future model work. Stronger DeBERTa-style QA baseline. Constrained decoding and validation. Retrieval and long-context evaluation later. Targeted ablations on chunk size, stride, prompt format, and negative sampling.

Final takeaway: LoRA fine-tuning improves this structured CUAD setup, with clear limits

Artifact package

Preprocessing summary, train summaries, metric reports, prediction samples, final plots, report, DOCX, and this deck.

Main conclusion

LoRA-based fine-tuning improves the structured CUAD extraction task compared with zero-shot prompting and the simple extractive QA baseline in this evaluated setup.

Bounded claim

The result is a reproducible graduate experiment, not a production legal system or direct CUAD-paper benchmark claim.

Q&A